

## Policies and Procedures Document

Andrew Smith  
Lewis Alexander  
Preben Rasamanickam  
Kyle Hannah

May 2026

# **1 Policies and Procedures Document**

## **1.0.1 Version Control and Code Management**

All project code was managed using a distributed version control system hosted on GitHub. The project was developed within a Dev Container environment, ensuring that all team members worked with identical toolchains and dependencies. Each team member developed features on individual branches, reducing the risk of conflicts and enabling parallel progress. Changes were merged into the main branch only after testing and verification had been demonstrated to the group. Frequent commits were encouraged to ensure that progress was continuously recorded and recoverable. This also enabled rollback to previously stable versions in the event of errors or regressions. Subsystem interfaces were defined prior to development, specifying expected input and output formats. This allowed consistent integration between components and ensured a uniform function naming convention across the codebase.

## **1.0.2 Code Review Practices**

Code reviews were conducted informally within the group prior to merging changes into the main branch. Reviews were performed after testing had been completed, ensuring that both functionality and code quality were assessed. Team members evaluated each other's work to identify potential issues, improve readability, and maintain consistency in coding style. Attention was given to critical components such as motor control and pathfinding logic, where incorrect behaviour could significantly impact system performance. This collaborative review process contributed to improved code reliability and reduced the likelihood of integration issues.

## **1.0.3 Task Allocation and Monitoring**

Tasks were allocated based on subsystem responsibilities, including motor control, sensor integration, state machine design, and pathfinding. This allowed each team member to focus on a specific area while maintaining clear ownership of their component. Hardware integration was treated as a shared responsibility and was carried out during the later stages of development, once most software components had been implemented. Progress was

monitored through weekly group meetings, where updates were shared and blockers identified. This ensured that issues were addressed early and that workload remained balanced across the team.

#### **1.0.4 Risk Management and Resilience**

To mitigate risks associated with delays or unforeseen issues, several measures were implemented. The use of version control ensured that all work was centrally accessible and protected against data loss. Clear documentation of subsystem interfaces allowed components to be understood and modified by multiple team members, reducing dependency on any single individual. Frequent commits ensured that development progress was regularly checkpointed, enabling recovery in the event of disruption. Regular integration and testing further ensured that the system remained functional throughout development, reducing the risk of late-stage failures.